

MAVIS · MINIMAX AGENT

The Skill Atlas

A complete operator's manual for every skill available in this environment — what each one does, when to load it, how to invoke it, and the prompts that actually work.

How skills work in this environment

Skills are modular capability packs that Mavis (this agent) can load on demand. Each one bundles a domain-specific workflow, instructions, scripts, and verification gates — instead of improvising every task from scratch, Mavis swaps in the right skill and follows its battle-tested playbook.

On-demand loading

Skills are not active by default. The `skill` tool pulls a skill's full `SKILL.md` into context on the first turn that needs it. After that, every instruction stays in scope until the session ends.

Trigger by intent

Load a skill when the user's request matches its description. Skip it for tiny edits, explanations, or work that doesn't fit the skill's narrow scope.

Strict step order

Skills like `deep-research` encode multi-step recipes. Skipping a step or reading later steps early is treated as an invalid trace.

Verify, then ship

Most skills ship with a verification gate — chart presence, image count, font fidelity, or schema validation. Run it before declaring "done."

The naming convention

Skill names follow the pattern `name` or `plugin:name`. The colon form means the skill belongs to a plugin; the plain form is a hosted skill loaded directly. Aliases (e.g. `docx` and `word:docx`) usually point at the same underlying capability — pick whichever is shorter or more memorable.

How to read each entry in this atlas

Every skill card in this document covers four angles: **(1)** what it actually does, **(2)** when to use it (and when *not* to), **(3)** exact invocation instructions, and **(4)** example prompts you can lift directly. Code blocks show real commands and JSON; copy them as-is.



Table of contents

00	Introduction · How skills work	03
01	Research & Planning	04
02	Documents	07
03	Engineering	13
04	Tools & Orchestration	17
05	Appendix · Cheatsheet & aliases	20

Research & Planning

Skills in this section handle open-ended investigation and pre-execution design. They trade raw speed for rigor: deep research forces a five-step trace with verification, plan-mode forces a structured conversation before any tool fires.

deep-research

deep-research

hosted · single-call skill

A five-step research pipeline for open-ended, source-dependent questions. Designed for market/industry analysis, technical research, competitor analysis, trend judgment, and policy/regulatory/fact verification. Each step prompt file **must** be read by an explicit Read tool call before that step starts — skipping or reordering steps is treated as an invalid trace.

What it does

The workflow is rigid: **Step 1** confirms factual background, **Step 2** understands the question and judges the direction, **Step 3** performs deep analysis and builds a research plan, **Step 4** searches, verifies, and forms the actual research understanding, **Step 5** writes the current-turn final answer file based on everything above. It is opinionated about citation, source hierarchy, and verification gates.

WHEN TO USE

- Market sizing or industry analysis
- Multi-source technical comparison
- Regulatory or policy surveys
- Trend judgments that need citations
- Any claim that must be defensible

WHEN NOT TO USE

- Casual factual recall ("what's the capital of France")
- Tiny one-line answers
- Tasks that need no source verification
- Real-time data — instead use a single `web_search`

How to invoke

```
skill(name="deep-research")
```

Example prompts

```
"Compare the current state of WebGPU adoption across the top 5 desktop browsers – include version cuts, fallback paths, and a sourced market share for each engine."
```

```
"Research the EU AI Act's high-risk classification criteria and map them to common generative-AI deployment patterns. Cite each criterion to its article number."
```

```
"I need a defensible market-size estimate for the on-device LLM inference chip market 2024-2028. Include bottom-up and top-down triangulation, name the sources."
```

Tip: If the user gives a single-source fact-check ("is X true?"), this skill is overkill — use `web_search` + `web_fetch` directly and skip the trace.

plan-mode

A "talk before you build" skill. Instead of executing immediately, Mavis enters plan mode: it surveys the request, surfaces the design decisions, raises hidden risks, and gets an aligned direction from the user — only then does it switch to execution mode and start making changes.

What it does

Identifies meaningful ambiguity, multiple valid approaches, or cases where the user explicitly asked to discuss first. Produces a structured plan: scope, options, trade-offs, open questions, recommended path. Pauses for confirmation before any tool with side effects runs.

WHEN TO USE

- Tasks with multiple valid approaches
- Architecture decisions that lock in cost
- User says "let's discuss" or "before you start..."
- Refactors with non-obvious blast radius
- Cross-cutting changes (deps, schema, config)

WHEN NOT TO USE

- Bug fixes with an obvious cause
- One-line edits or pure typo fixes
- Execution-stage clarity is already there

How to invoke

```
skill(name="plan-mode")
```

Example prompts

```
"I want to migrate the auth layer from session cookies  
to JWT. Before you write any code, lay out the migration  
plan: data-model diff, rollout strategy, rollback path."
```

```
"Plan how to add multi-tenant isolation to this Django  
app. Surface the design trade-offs (schema-per-tenant  
vs row-level vs app-level), then wait for my call."
```

```
"Let's redesign the homepage. Talk me through the layout  
options first — I want to pick the direction, not have  
you commit pixels."
```

Note: Plan-mode is for *design* decisions, not *codebase discovery*. If the question is "find where the bug lives," use `grep` / `glob` instead — don't waste a planning pass.

Documents

Skills for creating, editing, and reading formal documents — Word, PDF, PowerPoint, and Excel. Each one is the dedicated route for its format; cross-routing almost always loses fidelity. [visual-page](#) rounds out this group as the option when text alone can't carry the information.

docx · word:docx

docx

alias: `word:docx` · hosted

Unified DOCX skill — covers create, template-apply, edit/fill, read, repair, and compare Word documents. One skill to handle the entire .docx lifecycle instead of switching tools per task.

What it does

Provides workflows for: authoring a new Word doc from scratch (with template-aware styling), filling values into form-style Word documents, reading existing .docx contents into structured text, repairing damaged files, and diff-comparing two .docx revisions. Both spellings load the same underlying capability.

WHEN TO USE

- Formal reports, contracts, proposals in Word
- Editing an existing .docx without losing styles
- Comparing two versions of a Word doc
- Diagnosing a broken .docx

WHEN NOT TO USE

- PDF deliverables → use the `pdf` skill
- Slides → use the `pptx` skill
- Plain Markdown drafting (not formal Word)

How to invoke

```
skill(name="docx")  
# equivalent (alias):  
skill(name="word:docx")
```

Example prompts

```
"Generate a Word version of the project status report  
using the standard corporate template – keep the heading  
hierarchy, fill the body from this Markdown outline."
```

```
"Compare /contracts/2025-vendor.docx and  
/contracts/2026-vendor-renewed.docx and tell me which  
clauses changed and how."
```

```
"Open the attached .docx, extract every paragraph under  
section 4, and re-emit it as Markdown for editing."
```

Tip: Use the alias `word:docx` when you're working in a plugin context (e.g. an Office-related plugin); use plain `docx` anywhere else. Behavior is identical.

pdf · pdf:pdf

Unified PDF skill — generate, reformat, fill, and read PDFs. Covers text-to-PDF (reports, resumes, proposals, data-viz reports), Markdown → PDF conversion, PDF form filling, and PDF reading/extraction/OCR. Routes by user intent: CREATE (author from scratch), REFORMAT (restyle existing), FILL (write into form), READ (extract content), MUTATE (merge/split/rotate/sign).

What it does

Routes between five intents through a single SKILL.md entry: authoring a polished PDF from HTML+CSS+JS via Chromium, restyling Markdown to clean PDF, writing values into AcroForm fields, reading with pdfplumber (default) or vision (for scanned/chart PDFs), and qpdf/pypdf/reportlab for mutation. The cloud sandbox ships xelatex / latexmk for the LaTeX thesis and book routes.

WHEN TO USE

- Polished PDF deliverables (reports, resumes)
- Reading values out of an existing PDF
- Filling an AcroForm or visual-overlay PDF form
- Merge / split / rotate / sign / watermark
- LaTeX thesis or technical-book compile

WHEN NOT TO USE

- Just OCR-ing a single image (no PDF) → use native image reading
- Editing a Word .docx → use docx
- Generating a slide deck → use pptx
- Desktop-grade reflow / font work (out of scope)

How to invoke

```
skill(name="pdf")
# equivalent (alias):
skill(name="pdf:pdf")

# Render a polished report from HTML:
bash /workspace/.skills/pdf/scripts/make.sh render \
  --in page.html --out report.pdf --format A4 --wait 15000

# Read text from a PDF:
python3 -c "
import pdfplumber
with pdfplumber.open('in.pdf') as pdf:
    for p in pdf.pages: print(p.extract_text())
"
```

Example prompts

"Read the attached 60-page regulatory PDF and extract section 4.2 – stop at first heading index match, do NOT loop pdfplumber over every page."

"Take this Markdown quarterly-update and restyle it as a clean A4 PDF – preserve heading hierarchy, no chart recomposition."

"Fill /forms/w9.pdf with my data: John Doe, 123-45-6789, 123 Main St. AcroForm first, fall back to visual overlay if the form is flattened."

Mandatory rules: Always pass `--format A4` (or Letter) explicitly — Chromium silently overrides CSS `@page` if missing. Multi-page PDFs always need a clickable TOC. Don't suppress stderr — it's the only signal when something fails.

pptx · ppt:pptx

Read, create, and edit PowerPoint PPTX/PPT. Parses, summarizes, extracts content; inspects themes and layouts; creates new decks with PptxGenJS; edits existing PPTX while preserving formatting.

What it does

End-to-end .pptx lifecycle: parse an existing deck into text, summarize content, list themes and slide layouts, and either author a new deck from scratch (programmatic via PptxGenJS) or edit a deck in place while preserving master slides, fonts, and color palettes. Choose programmatically created decks for new content; choose editing when fidelity to an existing template matters.

WHEN TO USE

- Authoring a new slide deck from an outline
- Summarizing or extracting content from an existing deck
- Editing slides without losing the master template
- Inspecting themes/layouts for downstream style work

WHEN NOT TO USE

- PDF report → `pdf` skill
- Word doc → `docx` skill
- Keynote (Mac-only .key) — convert first, then come back

How to invoke

```
skill(name="pptx")  
# equivalent (alias):  
skill(name="ppt:pptx")
```

Example prompts

```
"Generate a 12-slide investor update deck — each slide  
follows the same template: title, three bullets, one chart.  
Use the brand colors from /brand/colors.json."
```

```
"Open /decks/q1-review.pptx, extract every speaker note,  
and re-emit it as a single Markdown brief."
```

```
"Edit /decks/sales-overview.pptx to add a 'Q3 Results'  
slide between slides 4 and 5 — keep the master template  
intact, do not break other slides."
```

Tip: "Preserve formatting" only works if you edit the existing file in place — do not regenerate from scratch when fidelity matters. Use PptxGenJS only for new decks.

excel:xlsx · xlsx

excel:xlsx

alias: `xlsx` · hosted

Spreadsheet skill — read, edit, create, and convert .xlsx/.xlsm/.csv/.tsv files. Triggered whenever a spreadsheet is the primary input or output: editing columns, formulas, formatting, charting, cleaning messy data, creating new spreadsheets from scratch.

What it does

Full .xlsx lifecycle: load and read tabular data, edit cells / formulas / formatting programmatically, build new spreadsheets with proper types (numbers vs text vs dates), create charts that survive Excel re-open, and convert between `xlsx` ↔ `csv` ↔ `tsv` ↔ `json`. Preserves formulas, named ranges, and conditional formatting where possible.

WHEN TO USE

- Editing columns / formulas in a workbook
- Cleaning messy data with mixed types
- Creating a new spreadsheet with formatting + chart
- Converting `xlsx` → `csv` / `json` / `tsv` or back

WHEN NOT TO USE

- Tabular data inside a PDF/Word/HTML deliverable
- CSV-only tasks that won't be opened in Excel
- Database tables — use SQL / DB tools, not a sheet

How to invoke

```
skill(name="excel:xlsx")  
# equivalent (alias):  
skill(name="xlsx")
```

Example prompts

```
"Open /data/sales-2025.xlsx, the 'Region' column has  
mixed case and trailing spaces – clean it, preserve all  
other columns and formulas, save in place."
```

```
"Build a new workbook with three sheets: 'Inputs',  
'Calculations', 'Output'. Use formulas across sheets, not  
hardcoded values. Add a bar chart to 'Output'."
```

```
"Convert the attached .xlsx to a clean CSV with UTF-8  
encoding, no quoting unless necessary, and LF line endings."
```

Note: The skill preserves formulas and types — but if you write values from Python via `.value =` you'll overwrite formulas with literals. Edit the formula strings, not the cell values, when you want calculations to live.

visual-page

Creates a self-contained visual HTML page (with diagrams, charts, tables, timelines, interactive layouts) when plain text cannot convey the information well. Use proactively when the content needs visual structure.

What it does

Generates a single HTML file with embedded CSS/JS — diagrams, charts (Chart.js or static SVG), tables, timelines, and interactive layouts — that the user can open directly in a browser. No build step, no deployment, no server. The output is portable: one file you can email, host, or drop into static storage.

WHEN TO USE

- Architecture diagrams or system-flow visuals
- Interactive timelines or dependency graphs
- Comparison tables that need sorting / filtering
- Any dense info-set that bullets can't carry

WHEN NOT TO USE

- Plain prose explanations
- Documents that need to be printable as PDF (use `pdf` instead)
- Multi-page docs (this is single-page by design)

How to invoke

```
skill(name="visual-page")
```

Example prompts

```
"Build me a single-page architecture diagram of this  
microservice stack with sub-second interactions on hover  
showing each service's responsibility."
```

```
"Make an interactive timeline of our roadmap for 2026 –  
quarters on the x-axis, milestones grouped by team,  
clickable to expand into a feature card."
```

```
"Create a sortable comparison table of the 8 candidate  
vector DBs we evaluated – columns: license, write perf,  
read perf, embedding size, hosting cost. Make it filterable."
```

Tip: If the user might want to print or share as a static artifact, the `pdf` skill is the better target. `visual-page` shines for browsers, screen recordings, and embedded mocks.

Engineering

The Senior Full-Stack Developer plugin family. `engineering-workflow` routes incoming work to the right layer; `frontend-dev` and `fullstack-dev` are the heavy-implementation entry points. Always load the workflow first when invoked via `@senior-fullstack-developer` .

senior-fullstack-developer:engineering-workflow

senior-fullstack-developer:engineering-workflow

plugin · primary entry point

The router skill. Load this first when the user invokes `@senior-fullstack-developer` or asks to build or implement a non-trivial frontend, backend, or full-stack feature. It provides repo-aware routing, incremental delivery, focused validation, and an internal quality audit.

What it does

Inspects the repo, classifies the work (frontend-only / backend-only / cross-layer), then dispatches to `frontend-dev`, `fullstack-dev`, or stays at the workflow level for small edits. It enforces incremental delivery — small, verifiable steps — and runs an internal quality audit at the end. Treat it as the contract for "do real engineering work," not the implementation itself.

WHEN TO USE

- User explicitly invokes `@senior-fullstack-developer`
- Non-trivial UI / API / DB / infra work in a repo
- Any change that needs focused validation + audit

WHEN NOT TO USE

- Single-line edits or copy changes
- Pure explanation tasks
- Formal code-review-only requests
- Non-coding work

How to invoke

```
skill(name="senior-fullstack-developer:engineering-workflow")
```

Example prompts

```
"@senior-fullstack-developer – build out the new
billing-dashboard page from this Figma. Repo is at
/workspace/billing-app. Use the existing component library."
```

```
"Hey – adding a /v2/search endpoint to our Python API. Use
the senior-fullstack workflow so we get proper validation
and a migration plan."
```

```
"I want to refactor the auth middleware. Run it through
the senior-fullstack engineering-workflow first so we
agree on the blast radius before changes."
```

Note: Always load this skill *first* when the user explicitly invokes `@senior-fullstack-developer` — it is the canonical entry and forwards to the correct implementation skill.

senior-fullstack-developer:frontend-dev

Implements substantial production frontend experiences. Loaded after `engineering-workflow` routes UI work, or when the user explicitly invokes this skill by name. Covers responsive layouts, accessibility, interaction states, motion systems, data-heavy UI, marketing pages, and local visual assets.

What it does

Codes production frontend against a real component system: responsive grids, a11y-correct focus/keyboard/ARIA semantics, motion choreography, large-data tables, marketing-page composition, and local asset handling. Treats the design system as load-bearing — never bypasses the existing tokens, never inlines raw hex values.

WHEN TO USE

- New full page or large component in a real app
- Marketing / landing pages with brand constraints
- Data-heavy tables, dashboards, infinite scroll
- Complex motion / animation choreography

WHEN NOT TO USE

- Tiny copy / color / spacing edits
- Backend-only work
- Visual critique without implementation
- Cross-layer concerns (use `fullstack-dev`)

How to invoke

```
skill(name="senior-fullstack-developer:frontend-dev")
```

Example prompts

```
"Build the analytics dashboard for our React app following the existing component library – make it responsive 320–1920px, keyboard-navigable, and lazy-load the chart bundle."
```

```
"Implement a new pricing-page landing experience matching the brand tokens at /brand. Hero animation should respect prefers-reduced-motion; the CTA needs real focus states – no invisible outlines."
```

```
"Convert this Figma into a Next.js route using the existing Card / Button primitives. Don't introduce new hex values – extend the design tokens instead."
```

Tip: For any visual critique ("is this layout good?"), pair this skill with `visual-page` to render mocked alternatives before committing.

senior-fullstack-developer:fullstack-dev

Implements substantial cross-layer software. Loaded after `engineering-workflow` routes full-stack work, or when the user explicitly invokes this skill by name. Covers frontend/backend contracts, APIs, authentication, databases, transactions, caching, background jobs, uploads, real-time, reliability, and production hardening.

What it does

Owns the seam between frontend and backend: contracts and types crossing the network, API surface design (REST/GraphQL/RPC), authentication and authorization flows, schema design and migrations, transactional correctness, caching strategies, background-job orchestration, uploads, real-time channels (WS/SSE), reliability patterns, and production hardening (retries, circuit breakers, observability).

WHEN TO USE

- New API + frontend feature in one pass
- Auth + user-model additions
- Schema migrations + transactional workflows
- Background-job / cron / webhook integration
- Real-time features (chat, presence, notifications)

WHEN NOT TO USE

- Tiny isolated edits
- Frontend-only visual work → use `frontend-dev`
- Pure explanation or docs requests
- Formal review-only tasks

How to invoke

```
skill(name="senior-fullstack-developer:fullstack-dev")
```

Example prompts

```
"Add SSO login to the web app – OAuth code-flow PKCE,
secure session cookies, server-side enforcement on every
API route, and a Postgres migration for the new tables.
Wire it through the existing login UI component."
```

```
"Build a file-upload pipeline: signed-URL direct upload,
background virus-scan job, thumbnail generation, and a
GET endpoint returning the asset. Include retry/backoff
on the scan job and an idempotency key on the API."
```

```
"Add real-time presence to our chat room – WS gateway,
Redis pub/sub fan-out, presence snapshots on reconnect,
heartbeats with timeout eviction. Spec before code."
```

Note: This skill explicitly owns the *contract* between layers. If you find yourself only writing React or only writing SQL, you may be in the wrong skill — switch to `frontend-dev` or a backend-only tool.

Tools & Orchestration

Skills that don't generate a deliverable themselves but unlock capabilities — App discovery via `mcode-tools-master` , parallel-agent orchestration via `team` , and turning repeated workflows into new skills via `skill-creator` .

mcode-tools-master

The discovery and invocation entry point for every App / Connector tool. The `mcode-tools` CLI is on PATH; this skill is the only way to discover, inspect, and call App tools. App tools are NOT injected into the model tool list — you must load this skill first.

What it does

Lists every connected App/Connector (Google Workspace, Notion, Slack, GitHub, etc.), describes their tool surface, and shows you how to call a specific App tool with arguments. It also exposes batch automation, scripts, and multimedia actions that aren't visible from the standard tool list. Ordinary MCP and Plugin-owned tools already in the model tool list can still be called directly without this skill.

WHEN TO USE

- User names a connected App (e.g. "post to Slack")
- Specialized external data behind a connector
- You need batch / scripting / multimedia actions
- Before **any** `mcode-tools` command in bash

WHEN NOT TO USE

- When an ordinary tool or MCP tool already covers the ask
- Generic web search → just call `web_search`
- Local filesystem / repo work → use Bash / Read / Edit directly

How to invoke

```
# Step 1 – load the skill (REQUIRED before any mcode-tools call)
skill(name="mcode-tools-master")

# Step 2 – list available apps / tools
mcode-tools list

# Step 3 – describe a specific app tool
mcode-tools describe

# Step 4 – call it
mcode-tools call --arg1=value1 --arg2=value2
```

Example prompts

"List the Notion apps and connectors we have access to, then create a new page in the 'Engineering' parent page using this Markdown body."

"Use the Google Sheets connector to read /spreadsheets/Q3-budget.xlsx – give me the total row of the 'Summary' sheet."

"Post a Slack message to #dev-announce with the release notes from /release-notes/2026-09-24.md, mention @channel in the body."

Mandatory: You must load this skill *before* any `mcode-tools` Bash command. Skipping this step is the most common cause of "tool not found" errors.

team

Runs a parallel team plan AND/OR creates the agents you need to do it. When one worker isn't enough, fan the same problem out across multiple specialized sub-agents, gather their outputs, and synthesize.

What it does

Defines a roster of sub-agents (each with a clear role + scope), launches them in parallel using `run_in_background: true`, collects their results via `task_query` / `task_output`, and synthesizes a unified final answer. Can also spin up new persistent agents via the `mavis` CLI when you need a new specialist for a recurring domain.

WHEN TO USE

- Complex multi-step research across domains
- Parallelizable investigation (multi-competitor scan, multi-jurisdiction review)
- Generation of a unified deliverable from several specialist outputs
- Long-running sub-tasks that would block the main thread

WHEN NOT TO USE

- Single deterministic task where you can do it directly
- Tasks where parallel sub-agents add coordination cost without parallelism benefit
- Reading one file / one grep — use the cheap tools

How to invoke

```
skill(name="team")

# Within the team skill, dispatch parallel sub-agents:
task(
  description="Competitor scan A",
  prompt="...",
  agent_name="researcher-A",
  run_in_background=true
)
task(
  description="Competitor scan B",
  prompt="...",
  agent_name="researcher-B",
  run_in_background=true
)
```

Example prompts

"Spin up a 3-agent team to compare these four vendors:
agent-1 does pricing, agent-2 does integration ergonomics,
agent-3 does compliance posture. Synthesize a unified
scorecard at the end."

"Use a parallel team to translate this 80-page regulatory
doc into Chinese, English, and Japanese simultaneously –
agents share a glossary but work independently."

"Background a 30-minute deep-research task on the entire
LLM inference market while I keep chatting with you about
something else."

Tip: Set `run_in_background: true` on any task expected to take more than ~1 minute. Poll with `task_query / task_output` ; stop with `task_stop` if it's clearly off-track.

skill-creator

Creates a new Mavis skill. Used when the user asks to create a skill, turn a repeated workflow into a skill, or build a new reusable procedure. Writes to `.skills/<name>/SKILL.md` for automatic sync at session end.

What it does

Takes a procedure you describe (or one you've been doing repeatedly in this session), authors a `SKILL.md` following the canonical skill format (description, operational rules, routes/components, references), and writes it under `/workspace/.skills/<name>/SKILL.md`. The skill syncer auto-detects new skills at session end and uploads them — after that, the skill is available to every future session.

WHEN TO USE

- User explicitly says "create a skill"
- A workflow is being repeated across multiple turns
- You keep rediscovering the same procedure from scratch
- You want a new capability to persist across sessions

WHEN NOT TO USE

- One-off task with no reuse potential
- Modifying existing skills (use Edit on `SKILL.md`)
- User is not the author of the environment — ask first

How to invoke

```
skill(name="skill-creator")
```

Example prompts

```
"Turn the workflow we've been using to triage customer support tickets into a reusable skill – name it 'support-triage' – and add it under /workspace/.skills/."
```

```
"Create a skill for our team's weekly release-note process: input (PR list), output (formatted Markdown release notes), and verification gate (all PRs have human-approved titles)."
```

```
"I keep rebuilding the same pytest coverage report HTML summary. Make it a skill called 'pytest-coverage-summary'."
```

Note: Skills you create here live in `/workspace/.skills/` and sync via the daemon. Delete with `bash mavis skill rm <name>` if a skill becomes stale or wrong.

Cheatsheet & aliases

Cheatsheet — pick the right skill in 10 seconds

IF THE TASK IS...	LOAD THIS SKILL	WHY
Multi-source research with citations	<code>deep-research</code>	Five-step trace with verification gate.
Big decision before any code runs	<code>plan-mode</code>	Surfaces options and waits for your call.
Authoring a Word document	<code>docx</code>	Preserves styles, headings, template fidelity.
Polished PDF from scratch	<code>pdf</code>	HTML → Chromium → PDF, design archetypes included.
Reading or filling an existing PDF	<code>pdf</code> (READ / FILL route)	pdfplumber default, vision for chart pages.
Slide deck (new or edit)	<code>pptx</code>	PptxGenJS for new, in-place edit to preserve format.
Spreadsheet work	<code>excel:xlsx</code>	Preserves formulas / types / conditional formatting.
Visual single-page artifact	<code>visual-page</code>	Self-contained HTML, ideal for diagrams / interactivity.
Substantial frontend implementation	<code>frontend-dev</code>	Owns a11y, motion, responsive, design tokens.
Cross-layer (API + auth + DB + UI)	<code>fullstack-dev</code>	Owns the seam between frontend and backend.
Calling a connected App (Slack, Notion...)	<code>mcode-tools-master</code>	Mandatory prerequisite for <code>mcode-tools</code> CLI.
Parallel multi-agent work	<code>team</code>	Fan out, fan in, synthesize with background tasks.
Turning a procedure into a reusable skill	<code>skill-creator</code>	Persists across sessions via skill syncer.

Aliases & duplicates

A few skills have alias names that map to the same underlying capability. Either spelling works; pick the shorter or more familiar one for your context.

PRIMARY NAME	ALIAS	NOTES
docx	word:docx	Plugin-prefixed alias; identical capability.
pdf	pdf:pdf	Plugin-prefixed alias; identical capability.
pptx	ppt:pptx	Plugin-prefixed alias; identical capability.
excel:xlsx	xlsx	Plain alias; identical capability.

Universal commands

```
# Load any skill (returns its full SKILL.md instructions)
skill(name="<skill-name>")

# Run a Bash command in the cloud sandbox
bash(command="echo hello", run_in_background=false)

# Read / Edit / Write files in the workspace
read(path="relative/path")
edit(file_path="...", old_string="...", new_string="...")
write(path="...", content="...")

# Web access — see "Hard Limits" in the root system prompt
web_search(query="...", count=10)
web_fetch(url="...", prompt="extract section 3.2")
```

End of Atlas

You now have a complete map of every skill in this environment — what each does, when to load it, how to invoke it, and the prompts that fit. Load skills on demand, verify their outputs before you ship, and reach for `skill-creator` when a workflow starts repeating itself.

For time-sensitive facts, the web is always the source of truth — use `web_search` / `web_fetch` directly and skip the heavy skills.